

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_2be29ecf-a31\agent-ab690a87225c63b82.jsonl`  
- SHA-256: `4b5db5fb054ca7ea80eaa9c292ac1dcd1f0e8f00b2452dfea5667b7209faabf3`  
- Source modified: `2026-06-22T01:28:08+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_2be29ecf-a31`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T01:26:03.398Z)

You are reviewing PR #310 "P2b-install" for DATA-LOSS / SAFETY bugs, especially scripts/uninstall.py.

Run `cd C:\\Users\\avidu\\Projects\\bhi-p2b && git diff origin/master...HEAD` and read scripts/uninstall.py + scripts/install.py (build_manifest, user_data) in C:\\Users\\avidu\\Projects\\bhi-p2b.

Scrutinize HARD: (1) Could uninstall.py delete USER DATA by default (it must KEEP config/.env/output unless --purge)? (2) Could --purge or the 'created' removal delete something it shouldn't (e.g. the whole app-home, the running script itself, a path outside app-home, follow a symlink)? (3) Does _rm guard against rmtree on a symlinked dir / a too-broad path? (4) Could the manifest's 'created' list ever contain a user-data path (so a non-purge uninstall deletes data)? (5) Does the no-overwrite config seeding protect a user's existing config + the future wizard's choices? Treat any path-deletion as guilty-until-proven-safe. Report concrete findings + fixes. Empty list if nothing real.

2. assistant (2026-06-22T01:26:06.467Z)

I'll review PR #310 for data-loss and safety bugs. Let me start by pulling the diff and reading the relevant files.

3. user (2026-06-22T01:26:09.771Z)

```
---BRANCH---  
feat/pkg-p2b-install  
---HEAD---  
78e423d docs(pkg): link P2b-install PR #310 in the §7 tracker
```

4. assistant (2026-06-22T01:26:14.969Z)

Let me read the diff and the key files.

5. user (2026-06-22T01:26:15.942Z)

```
README.md | 31 ++++++  
backend/config/__init__.py | 9 +-  
backend/config/loader.py | 34 ++++++  
backend/main.py | 15 +++  
docs/PACKAGING_PLAN.md | 13 +-  
scripts/install.py | 210 ++++++  
scripts/uninstall.py | 150 ++++++  
tests/test_app_launcher.py | 42 ++++++  
tests/test_p2b_install.py | 163 ++++++  
9 files changed, 660 insertions(+), 7 deletions(-)
```

6. user (2026-06-22T01:26:19.100Z)

```
1      #!/usr/bin/env python3
2      """Clean uninstaller for the KhelSutra highlight indexer (PACKAGING_PLAN.md P2b).
3
4      Reverses ``scripts/install.py`` from the manifest it wrote:
5
6      * uninstalls the package (``uv tool uninstall`` / ``pip uninstall -y`` ? matches the
install method);
7      * removes the files the installer created (launcher shortcut, the copied uninstaller,
the manifest);
8      * by default **KEEPS your data** (the seeded ``config.json``, your ``.env`` with the
Gemini key,
9      and the ``output/`` reels + DB). Pass ``--purge`` to delete those too.
10
11      Usage::
12
13          python uninstall.py                # find the manifest in the app-home, keep user
data
14          python uninstall.py --purge        # also delete config.json / .env / output (your
data)
15          python uninstall.py --manifest /path/to/uninstall_manifest.json
16          python uninstall.py --dry-run      # print the plan, change nothing
17
18      stdlib-only; safe to run from the copy the installer dropped in the app-home.
19      """
20      from __future__ import annotations
21
22      import argparse
23      import json
24      import os
25      import platform
26      import shutil
27      import subprocess
28      import sys
29      from pathlib import Path
30
31      APP_DIR_NAME = "Khelsutra"
32      MANIFEST_NAME = "uninstall_manifest.json"
33
34
35      def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
36                             env: dict[str, str] | None = None) -> Path:
37          """Mirror of the installer's app-home resolver (used to LOCATE the manifest when not
given)."""
38          system = system or platform.system()
39          e = os.environ if env is None else env
40          if system == "Windows":
41              base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
42          elif system == "Darwin":
43              base = str(Path.home() / "Library" / "Application Support")
44          else:
45              base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
46          return Path(base) / app_name
47
48
49      def find_manifest(explicit: str | None) -> Path | None:
50          """The manifest path: explicit arg, else next to THIS file, else the default app-
home."""
51          if explicit:
52              p = Path(explicit)
53              return p if p.exists() else None
54          for cand in (Path(__file__).resolve().parent / MANIFEST_NAME,
55                     default_os_app_home() / MANIFEST_NAME):
56              if cand.exists():
```

```

57         return cand
58     return None
59
60
61 def uninstall_command(method: str, package: str) -> list[str]:
62     """The argv to remove the package (PURE)."""
63     if method == "uv":
64         return ["uv", "tool", "uninstall", package]
65     return [sys.executable, "-m", "pip", "uninstall", "-y", package]
66
67
68 def _rm(path: Path, dry: bool) -> None:
69     try:
70         if path.is_dir() and not path.is_symlink():
71             print(f" - rmtree {path}")
72             if not dry:
73                 shutil.rmtree(path, ignore_errors=True)
74         elif path.exists() or path.is_symlink():
75             print(f" - remove {path}")
76             if not dry:
77                 path.unlink(missing_ok=True)
78     except OSError as e:
79         print(f" ! could not remove {path}: {e}")
80
81
82 def main(argv: list[str] | None = None) -> int:
83     ap = argparse.ArgumentParser(description="Uninstall the KhelSutra highlight
indexer.")
84     ap.add_argument("--manifest", default=None, help="Path to uninstall_manifest.json.")
85     ap.add_argument("--purge", action="store_true",
86                     help="Also delete YOUR data (config.json, .env, output/). Default
keeps it.")
87     ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
nothing.")
88     args = ap.parse_args(argv)
89
90     manifest_path = find_manifest(args.manifest)
91     if not manifest_path:
92         print("[ERROR] no uninstall_manifest.json found (pass --manifest <path>).")
93         return 1
94     manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
95     method = manifest.get("install_method", "pip")
96     package = manifest.get("package", "badminton-highlight-indexer")
97     created = [Path(p) for p in manifest.get("created", [])]
98     user_data = [Path(p) for p in manifest.get("user_data", [])]
99     app_home = Path(manifest.get("app_home", str(manifest_path.parent)))
100
101     print(f"KhelSutra uninstaller ? {package} (installed via {method})")
102     print(f" manifest : {manifest_path}")
103     print(f" app-home : {app_home}")
104     print(f" data      : {'WILL BE DELETED (--purge)' if args.purge else 'kept (use
--purge to delete)'}")
105     dry = args.dry_run
106
107     print("\n[1/3] removing the package?")
108     cmd = uninstall_command(method, package)
109     print(f" $ {' '.join(cmd)}")
110     if not dry:
111         try:
112             subprocess.run(cmd, check=False)
113         except FileNotFoundError as e:
114             print(f" ! {method} not found ({e}); remove the package manually.")
115
116     print("\n[2/3] removing installer-created files?")
117     # Remove the manifest LAST (we still reference it); skip the running uninstaller

```

```

copy itself.
118     self_path = Path(__file__).resolve()
119     for p in created:
120         if p.resolve() == self_path or p.resolve() == manifest_path.resolve():
121             continue
122         _rm(p, dry)
123
124     if args.purge:
125         print("\n[3/3] purging user data (--purge)?")
126         for p in user_data:
127             _rm(p, dry)
128     else:
129         print("\n[3/3] keeping your data:")
130         for p in user_data:
131             print(f"  · {p}")
132
133     # Manifest + (optionally) the empty app-home last.
134     _rm(manifest_path, dry)
135     if args.purge and app_home.exists():
136         try:
137             remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
138             if not remaining:
139                 _rm(app_home, dry)
140         except OSError:
141             pass
142
143     print("\n? Uninstalled." + (" " if args.purge else " Your data was kept."))
144     if self_path.parent == app_home:
145         print(f"    (you can delete this script + the app-home folder manually:
146 {app_home})")
147     return 0
148
149 if __name__ == "__main__":
150     raise SystemExit(main())
151

```

7. user (2026-06-22T01:26:19.406Z)

```

1     #!/usr/bin/env python3
2     """Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md
3 P2b).
4     Cross-platform (Windows / macOS / Linux), no native component (01):
5
6     * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
isolated tool
7     env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
8     * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data
dir and runs
9     ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
10    (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the
process
11    starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env
set by the
12    launcher (not by the engine) roots the ``.env`` in the app-home;
13    * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.
14
15    The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
16    ``indexer --app`` run (``write_light_default_config``), so it works regardless of
install method;
17    this script only wires the install + the launcher.
18
19    Usage::
20

```

```

21         python scripts/install.py                    # install '.' (this checkout), uv-
if-present else pip
22         python scripts/install.py --target badminton-highlight-indexer # from PyPI (once
published)
23         python scripts/install.py --method pip        # force pip
24         python scripts/install.py --dry-run           # print the plan, change nothing
25
26     stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
27     """
28     from __future__ import annotations
29
30     import argparse
31     import json
32     import os
33     import platform
34     import shutil
35     import subprocess
36     import sys
37     from datetime import datetime, timezone
38     from pathlib import Path
39
40     PACKAGE = "badminton-highlight-indexer"
41     APP_DIR_NAME = "Khelsutra" # mirrors backend.utils.app_home.APP_DIR_NAME
42     MANIFEST_NAME = "uninstall_manifest.json"
43
44
45     # --- pure helpers (unit-tested)
-----
46
47     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
48                             env: dict[str, str] | None = None) -> Path:
49         """The OS-conventional per-user app-data dir ? a standalone mirror of
50         ``backend.utils.app_home.default_os_app_home`` (this script runs before the package
exists)."""
51         system = system or platform.system()
52         e = os.environ if env is None else env
53         if system == "Windows":
54             base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
55         elif system == "Darwin":
56             base = str(Path.home() / "Library" / "Application Support")
57         else:
58             base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
59         return Path(base) / app_name
60
61
62     def shortcut_spec(system: str, app_home: Path, indexer_cmd: str = "indexer") ->
tuple[str, str, bool]:
63         """Return ``(filename, content, make_executable)`` for the per-OS launcher shortcut.
64
65         The shortcut pins ``RALLY_APP_HOME`` then runs ``indexer --app``. PURE (no IO) so
it's testable.
66         """
67         home = str(app_home)
68         if system == "Windows":
69             content = (
70                 "@echo off\r\n"
71                 "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens
the app.\r\n"
72                 f'set "RALLY_APP_HOME={home}"\r\n'
73                 f'{indexer_cmd} --app\r\n'
74             )
75             return "KhelSutra.bat", content, False
76         if system == "Darwin":
77             content = (
78                 "#!/bin/bash\n"

```

```

79         "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
80         f'export RALLY_APP_HOME="{home}"\n'
81         f"exec {indexer_cmd} --app\n"
82     )
83     return "KhelSutra.command", content, True
84 # Linux / other
85 content = (
86     "#!/bin/bash\n"
87     "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
88     f'export RALLY_APP_HOME="{home}"\n'
89     f"exec {indexer_cmd} --app\n"
90 )
91 return "khelsutra.sh", content, True
92
93
94 def build_manifest(*, method: str, app_home: Path, shortcut: Path, created: list[Path],
95                  timestamp: str) -> dict:
96     """The uninstall manifest (PURE). ``created`` = files THIS installer made (safe to
remove);
97     ``user_data`` = paths the engine/user own (kept unless ``uninstall.py --purge``)."""
98     return {
99         "schema": 1,
100        "product": "KhelSutra highlight indexer",
101        "package": PACKAGE,
102        "install_method": method,          # "uv" | "pip"
103        "indexer_command": "indexer",
104        "app_home": str(app_home),
105        "shortcut": str(shortcut),
106        "created": [str(p) for p in created],
107        "user_data": [
108            str(app_home / "config.json"),
109            str(app_home / ".env"),
110            str(app_home / "output"),
111        ],
112        "timestamp": timestamp,
113    }
114
115
116 def resolve_method(requested: str, uv_present: bool) -> str:
117     """Pick the installer: explicit beats auto; auto prefers uv when present, else
pip."""
118     if requested in ("uv", "pip"):
119         return requested
120     return "uv" if uv_present else "pip"
121
122
123 def install_command(method: str, target: str) -> list[str]:
124     """The argv for the chosen installer (PURE)."""
125     if method == "uv":
126         return ["uv", "tool", "install", target]
127     return [sys.executable, "-m", "pip", "install", target]
128
129
130 # --- IO / orchestration
-----
131
132 def _run(argv: list[str]) -> None:
133     print(f" $ {' '.join(argv)}")
134     subprocess.run(argv, check=True)
135
136
137 def main(argv: list[str] | None = None) -> int:
138     ap = argparse.ArgumentParser(description="Install the KhelSutra highlight indexer

```

```

(LIGHT tier).")
139     ap.add_argument("--target", default=".",
140                     help="What to install: '.' (this checkout, default) or a PyPI
name/spec.")
141     ap.add_argument("--method", choices=("auto", "uv", "pip"), default="auto",
142                     help="Installer to use (default: uv if present, else pip).")
143     ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
nothing.")
144     args = ap.parse_args(argv)
145
146     system = platform.system()
147     app_home = default_os_app_home()
148     method = resolve_method(args.method, shutil.which("uv") is not None)
149     cmd = install_command(method, args.target)
150     fname, content, executable = shortcut_spec(system, app_home)
151     shortcut = app_home / fname
152
153     print(f"KheISutra installer ? {PACKAGE} (LIGHT, torch-free)")
154     print(f" OS           : {system}")
155     print(f" installer    : {method}")
156     print(f" target       : {args.target}")
157     print(f" app-home     : {app_home}")
158     print(f" shortcut     : {shortcut}")
159     if args.dry_run:
160         print("\n[dry-run] would run:")
161         print(f" $ {' '.join(cmd)}")
162         print(f" write shortcut -> {shortcut}")
163         print(f" write manifest -> {app_home / MANIFEST_NAME}")
164         print(f" copy uninstaller -> {app_home / 'uninstall.py'}")
165         return 0
166
167     print("\n[1/3] installing the package?")
168     try:
169         _run(cmd)
170     except (subprocess.CalledProcessError, FileNotFoundError) as e:
171         print(f"[ERROR] install failed: {e}")
172         if method == "uv":
173             print(" (uv not usable? re-run with --method pip)")
174         return 1
175
176     print("\n[2/3] writing the launcher shortcut + app-home?")
177     app_home.mkdir(parents=True, exist_ok=True)
178     shortcut.write_text(content, encoding="utf-8")
179     if executable:
180         shortcut.chmod(0o755)
181     # Make the uninstaller available next to the data it removes.
182     created: list[Path] = [shortcut]
183     here = Path(__file__).resolve().parent
184     src_uninstall = here / "uninstall.py"
185     dst_uninstall = app_home / "uninstall.py"
186     if src_uninstall.exists():
187         shutil.copyfile(src_uninstall, dst_uninstall)
188         created.append(dst_uninstall)
189
190     print("\n[3/3] writing the uninstall manifest?")
191     manifest_path = app_home / MANIFEST_NAME
192     created.append(manifest_path)
193     manifest = build_manifest(
194         method=method, app_home=app_home, shortcut=shortcut, created=created,
195         timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
196     )
197     manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
198
199     print("\n? Installed. To launch KheISutra:")
200     if system == "Windows":

```

```

201         print(f"    double-click {shortcut}    (or run `indexer --app`)")
202     else:
203         print(f"    {shortcut}    (or run `indexer --app`)")
204     print("    The first run seeds a truly-local 'motion' config (offline, no key/GPU
needed).")
205     print(f"    To uninstall: python \"{dst_uninstall}\"    (add --purge to also delete
your data)")
206     return 0
207
208
209     if __name__ == "__main__":
210         raise SystemExit(main())
211

```

8. assistant (2026-06-22T01:26:22.335Z)

Now let me look at the config seeding logic and the engine-side app_home resolver to verify consistency and check the no-overwrite config seeding.

9. user (2026-06-22T01:26:23.352Z)

```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,
26         preset_for,
27         suggest_profile,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
it honours
33     # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
relative) when
34     # the env is unset, exactly the pre-P0a value. (The old module-level
``DEFAULT_CONFIG_PATH`` constant
35     # was dropped: it had no external consumers and a frozen-at-import snapshot would
silently diverge
36     # from a later RALLY_APP_HOME.)
37
38
39     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
40         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,

```



```

scalars
41         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
42         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
43         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
44         out = dict(base)
45         for key, val in override.items():
46             if isinstance(val, dict) and isinstance(out.get(key), dict):
47                 out[key] = _deep_merge(out[key], val)
48             else:
49                 out[key] = val
50         return out
51
52
53     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
54         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
55         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
56         environment never silently selects a profile (and never silently spends on cloud).
57
58         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
59         profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
60         asserting a preset that was never applied. A bad value in config.json is left to
surface as
61         a hard, typed-Literal error downstream (config-file correctness)."""
62         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
63         if env_profile and env_profile.strip():
64             ep = env_profile.strip()
65             if is_known_profile(ep):
66                 return ep
67             logger.warning(
68                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
69                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
70             )
71             # fall through to the file's profile (env typo must not suppress a valid file
choice)
72         dep = file_data.get("deployment")
73         if isinstance(dep, dict):
74             prof = dep.get("profile")
75             if isinstance(prof, str) and prof.strip():
76                 return prof.strip()
77         return ""
78
79
80     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
81         """Dotted paths in `data` that the typed config will not honor.
82
83         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
84         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
85         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
86         user's intent is lost ? collect every such key so load_config can warn.
87         """
88         unknown: list[str] = []
89         fields = model_cls.model_fields
90         for key, val in data.items():
91             if key not in fields:
92                 unknown.append(prefix + key)
93                 continue
94             ann = fields[key].annotation

```

```

95         if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
96             unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
97     return unknown
98
99
100 def _get_builtin_defaults() -> dict[str, Any]:
101     """Return a plain dict of the current built-in defaults.
102
103     The AppConfig model is the single source of truth for defaults.
104     """
105     return AppConfig().model_dump(mode="json")
106
107
108 def load_config(path: str | Path | None = None) -> AppConfig:
109     """Load configuration with the following precedence:
110
111     1. Built-in defaults defined in the Pydantic models.
112     2. Values from the JSON file (if present and readable).
113     3. (Future) environment / CLI overrides.
114
115     Missing file or unreadable file ? returns a fully defaulted AppConfig.
116     Unknown keys in the file are tolerated (extra="allow" on the model).
117     """
118     cfg_path = Path(path) if path is not None else default_config_path()
119
120     file_data: dict[str, Any] = {}
121     if cfg_path.exists():
122         try:
123             with open(cfg_path, "r", encoding="utf-8") as f:
124                 file_data = json.load(f)
125         except Exception as exc:
126             # Non-fatal: continue with defaults + whatever we could parse.
127             # In production we might log here.
128             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
129
130     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
131     # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
132     # non-mapping breaks the `**defaults, **file_data` unpack. Degrade to defaults +
warn, per the
133     # loader's "bad input is non-fatal" contract.
134     if not isinstance(file_data, dict):
135         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
136                        cfg_path, type(file_data).__name__)
137         file_data = {}
138
139     if file_data:
140         unknown = _unknown_key_paths(file_data, AppConfig)
141         if unknown:
142             logger.warning(
143                 "[CONFIG WARNING] %s contains keys the typed config does not "
144                 "recognize (typo'd or removed knobs ? top-level extras are kept "
145                 "but unread; unknown section keys are silently dropped): %s",
146                 cfg_path, ", ".join(sorted(unknown)),
147             )
148
149     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
150     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
151     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).

```

```

With no
152     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
153     defaults = _get_builtin_defaults()
154     profile = _resolve_explicit_profile(file_data)
155
156     if profile and not is_known_profile(profile):
157         # Only an unrecognised value from config.json reaches here (env typos already
fell back
158         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
159         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
160         logger.warning(
161             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
162             profile, ", ".join(PROFILE_PRESETS),
163         )
164         profile = ""
165
166     if profile:
167         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
168         # Record the profile actually applied ? the env may have chosen it over
config.json,
169         # so re-stamp it onto the merged deployment section to keep the record honest.
170         merged.setdefault("deployment", {})
171         if isinstance(merged["deployment"], dict):
172             merged["deployment"]["profile"] = profile
173             ct = merged["deployment"].get("compute_target")
174             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
175             if not ct and canonical:
176                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
177                 # to "") gets its canonical target back ? the record never silently lies
about an
178                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
179                 merged["deployment"]["compute_target"] = canonical
180             elif ct and canonical and ct != canonical:
181                 logger.warning(
182                     "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
183                     "config.json overrides it to %r ? honoring the explicit override.",
184                     profile, canonical, ct,
185                 )
186                 logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
187                             profile, merged["deployment"].get("compute_target"))
188     else:
189         # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
190         merged = {**defaults, **file_data}
191         suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
192         if suggestion:
193             logger.debug(
194                 "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
195                 "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
196                 "auto-applies ? D15).", suggestion,
197             )
198
199     return AppConfig.model_validate(merged)
200
201

```

```

202     def save_default_config(path: str | Path | None = None) -> Path:
203         """Write a fresh default config.json.
204
205         Used by scripts/doctor.py diagnostics and the --setup CLI path.
206         Overwrites if the file exists.
207         """
208         cfg_path = Path(path) if path is not None else default_config_path()
209         cfg = AppConfig()
210         cfg_path.parent.mkdir(parents=True, exist_ok=True)
211         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
212         return cfg_path
213
214
215     # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
216     # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
217     skip_ai_handoff,
218     # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
219     technical
220     # user gets ? a torch-free, offline, zero-setup experience:
221     # * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key
222     (LIGHT tier);
223     # * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-
224     bills on
225     Gemini nor returns 0 rallies on WASB-without-a-key
226     (08 intent);
227     # * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it
228     either way).
229     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
230     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
231         "sport": "badminton",
232         "deployment": {"profile": "truly-local"},
233         "indexing": {"default_segmenter": "motion", "use_gpu": False},
234     }
235
236
237     def write_light_default_config(
238         path: str | Path | None = None, *, overwrite: bool = False
239     ) -> tuple[Path, bool]:
240         """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
241
242         Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL
243         AppConfig
244         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
245         default ? **never
246         overwrites an existing file**, so a user's edits or the first-run wizard's choices
247         are preserved
248         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
249         dirs.
250         """
251         cfg_path = Path(path) if path is not None else default_config_path()
252         if cfg_path.exists() and not overwrite:
253             return cfg_path, False
254         cfg_path.parent.mkdir(parents=True, exist_ok=True)
255         cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n",
256                             encoding="utf-8")
257         return cfg_path, True
258
259
260     # Backwards-compatible alias used by some older call sites during transition.
261     def get_default_config() -> dict[str, Any]:
262         """Return defaults as a plain dict (for transitional use only)."""
263         return _get_builtin_defaults()
264
265

```

```
diff --git a/backend/config/__init__.py b/backend/config/__init__.py
index 9b276c0..6c38999 100644
--- a/backend/config/__init__.py
+++ b/backend/config/__init__.py
@@ -6,7 +6,12 @@ Public API:
 This package is the canonical place for all configuration access.
 """

-from .loader import load_config, save_default_config
+from .loader import (
+    LIGHT_DEFAULT_CONFIG,
+    load_config,
+    save_default_config,
+    write_light_default_config,
+)
 from .models import ( # re-export common ones
     AppConfig,
     Config,
@@ -27,6 +32,8 @@ from .startup import (
 __all__ = [
     "load_config",
     "save_default_config",
+    "write_light_default_config",
+    "LIGHT_DEFAULT_CONFIG",
     "AppConfig",
     "Config",
     "DeploymentConfig",
diff --git a/backend/config/loader.py b/backend/config/loader.py
index 37ed101..274c4f7 100644
--- a/backend/config/loader.py
+++ b/backend/config/loader.py
@@ -212,6 +212,40 @@ def save_default_config(path: str | Path | None = None) -> Path:
     return cfg_path

+# The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
+# Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies skip_ai_handoff,
+# local storage and the compute_target, so this stays the handful of knobs a fresh, non-technical
+# user gets ? a torch-free, offline, zero-setup experience:
+# * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key (LIGHT tier);
+# * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-bills on Gemini nor returns 0 rallies on WASB-without-a-key (08 intent);
+# * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it either way).
+# The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
+LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
+    "sport": "badminton",
+    "deployment": {"profile": "truly-local"},
+    "indexing": {"default_segmenter": "motion", "use_gpu": False},
+}
+
+def write_light_default_config(
+    path: str | Path | None = None, *, overwrite: bool = False
+) -> tuple[Path, bool]:
+    """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
+
+    Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL AppConfig and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by default ? **never overwrites an existing file**, so a user's edits or the first-run wizard's choices are
```

```

preserved
+ (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent dirs.
+ """
+ cfg_path = Path(path) if path is not None else default_config_path()
+ if cfg_path.exists() and not overwrite:
+     return cfg_path, False
+ cfg_path.parent.mkdir(parents=True, exist_ok=True)
+ cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8")
+ return cfg_path, True
+
+
# Backwards-compatible alias used by some older call sites during transition.
def get_default_config() -> dict[str, Any]:
    """Return defaults as a plain dict (for transitional use only)."""
diff --git a/backend/main.py b/backend/main.py
index c1738b4..4e5a19a 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -48,6 +48,7 @@ from backend.utils.single_instance import acquire_lock # P0e: one instance
per
from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
from backend.config import ( # new typed config
    load_config as load_app_config,
+ write_light_default_config, # P2b: batteries-included first-run config seeding
    AppConfig,
    StitchingConfig,
    enforce_startup_checks,
@@ -934,6 +935,20 @@ def main():
    run_cli_diagnose_clip(args.video_path, args.debug_clip)
    return

+ # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly `--app`
+ # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
+ # no API key, no GPU, no cloud bills) at the resolved app-home (default_config_path(),
which
+ # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE load_app_config
reads
+ # it ? so a fresh `pip`/`uv` tool install + `indexer --app` just works. NEVER overwrites an
+ # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped to
--app
+ # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
+ if args.app:
+     seeded_path, seeded = write_light_default_config()
+     if seeded:
+         print(f"[APP] First run: seeded a truly-local default config at {seeded_path} "
+             f"(detector 'motion' ? offline, no API key or GPU needed). "
+             f"Use the in-app setup to switch to Gemini.")
+
+ # Initialize Global Config and DB
global global_config, db_instance
global_config = load_app_config() # returns typed AppConfig

```

11. assistant (2026-06-22T01:26:28.534Z)

Now let me examine the engine-side `app\_home.py` to verify the manifest's `user\_data` paths are accurate and complete (e.g., is the DB inside `output/` or directly in app-home?), and to verify the resolver mirrors match.

12. user (2026-06-22T01:26:29.612Z)

```

1     """Resolved application-data HOME for a packaged, batteries-included install.
2
3     PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,

```

```

4     the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5     ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6     the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
7     "loses" their index. This module introduces ONE resolver every state path roots on.
8
9     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10    * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
sets it).
11    * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13    The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/local/share``)
14    is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15    calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16    this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17    real user profile during a test run).
18
19    stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20    the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21    """
22
23    from __future__ import annotations
24
25    import os
26    import platform
27    from pathlib import Path
28
29    #: Environment variable the packaged launcher sets to pin the app-data home.
30    APP_HOME_ENV = "RALLY_APP_HOME"
31
32    #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33    APP_DIR_NAME = "Khelsutra"
34
35
36    def app_home() -> Path:
37        """The resolved application-data home.
38
39        ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40        to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41        """
42        env = os.environ.get(APP_HOME_ENV)
43        if env and env.strip():
44            return Path(env.strip()).expanduser()
45        return Path(".")
46
47
48    def app_path(*parts: str | os.PathLike[str]) -> Path:
49        """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50        return app_home().joinpath(*[str(p) for p in parts])
51
52
53    def default_config_path() -> Path:
54        """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56        With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),

```

```

unchanged.
57         """
58         return app_home() / "config.json"
59
60
61     def default_env_path() -> Path:
62         """`<app-home>/env` ? the dotenv file the server prefers (e.g.
63         ``GEMINI_API_KEY``)."""
64         return app_home() / ".env"
65
66     def env_file_to_load() -> Path | None:
67         """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
68         DEFAULT
69         (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
70         behaviour.
71         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
72         exists. With the
73         env UNSET this returns ``None`` so the caller keeps the historical search and does
74         NOT newly
75         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
76         not the CWD).
77         """
78         home = app_home()
79         if str(home) == ".": # env unset (or pinned to CWD) ? historical load_dotenv()
80             search
81             return None
82             candidate = home / ".env"
83             return candidate if candidate.exists() else None
84
85     def resolve_output_dir(output_dir: str) -> str:
86         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
87         through.
88         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
89         ``"output"`` when
90         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
91         pathlib
92         normalisation of ``/x`` / separators / trailing slash). With it SET, a relative dir
93         like
94         ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
95         rest of the
96         app's state; an absolute path ? or a Windows drive-anchored one (``C:\x`` /
97         ``C:x``) ? passes
98         through untouched.
99         """
100         home = app_home()
101         if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
102             return output_dir or "output"
103         if not output_dir:
104             return str(home / "output")
105         if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
106             return output_dir
107         return str(home / output_dir)
108
109     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
110         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
111         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
112
113         * Windows ? ``%APPDATA%\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
114         * macOS ? ``~/Library/Application Support/<app_name>``
115         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if

```



```

unset)
108     """
109     system = platform.system()
110     if system == "Windows":
111         base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
112     elif system == "Darwin":
113         base = str(Path.home() / "Library" / "Application Support")
114     else:
115         base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
116     return Path(base) / app_name
117

```

13. assistant (2026-06-22T01:26:33.820Z)

Now let me check where the SQLite DB actually lives. The manifest's `user\_data` lists `config.json`, `.env`, and `output/`. I need to verify the DB is inside `output/` (otherwise a non-purge uninstall is fine but purge would leave the DB; or worse, the DB lives elsewhere). Let me check the DB path resolution.

14. user (2026-06-22T01:26:36.255Z)

```

25:     def __init__(self, db_path: str):
26:         self.db_path = db_path
47:         conn = sqlite3.connect(self.db_path, timeout=30.0)
113:             f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}")
150:         backup = f"{self.db_path}.bak-v{from_version}"
153:         shutil.copy2(self.db_path, backup)
168:         shutil.copy2(self.db_path, dest)

```

15. user (2026-06-22T01:26:36.393Z)

```

1     """P2b-install (PACKAGING_PLAN.md): batteries-included config seeding +
install/uninstall tooling.
2
3     Covers:
4     * the engine-side ``write_light_default_config`` (gate 08 default = truly-local +
motion), incl.
5     the load-bearing **no-overwrite** idempotency and the EFFECTIVE config the loader
produces;
6     * the pure logic of ``scripts/install.py`` / ``scripts/uninstall.py`` (shortcut
content, manifest,
7     installer/uninstaller argv, method resolution, OS app-home) ? no real subprocess/IO.
8     """
9     from __future__ import annotations
10
11     import importlib.util
12     import json
13     from pathlib import Path
14
15     import pytest
16
17     from backend.config import LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config
18
19
20     def _load_script(name: str):
21         """Load a scripts/*.py bootstrap file as a module (it is stdlib-only, safe to
exec)."""
22         path = Path(__file__).resolve().parents[1] / "scripts" / name
23         spec = importlib.util.spec_from_file_location(name[:-3], path)
24         assert spec and spec.loader
25         mod = importlib.util.module_from_spec(spec)
26         spec.loader.exec_module(mod)
27         return mod
28

```

```

29
30     # --- engine seeding: write_light_default_config (gate 08)
31     -----
32     def test_seed_writes_truly_local_motion(tmp_path):
33         cfg = tmp_path / "sub" / "config.json" # also proves parent-dir creation
34         path, written = write_light_default_config(cfg)
35         assert written is True and path == cfg and cfg.exists()
36         data = json.loads(cfg.read_text(encoding="utf-8"))
37         assert data == LIGHT_DEFAULT_CONFIG
38         assert data["indexing"]["default_segmenter"] == "motion"
39         assert data["indexing"]["use_gpu"] is False
40         assert data["deployment"]["profile"] == "truly-local"
41
42
43     def test_seed_does_not_overwrite_existing(tmp_path):
44         cfg = tmp_path / "config.json"
45         cfg.write_text('{"sport": "tennis", "indexing": {"default_segmenter": "gemini"}}',
46 encoding="utf-8")
47         path, written = write_light_default_config(cfg)
48         assert written is False and path == cfg
49         # The user's file is preserved untouched (the wizard/edits are never clobbered).
50         assert json.loads(cfg.read_text(encoding="utf-8"))["indexing"]["default_segmenter"]
51 == "gemini"
52
53     def test_seed_overwrite_true_forces(tmp_path):
54         cfg = tmp_path / "config.json"
55         cfg.write_text("{}", encoding="utf-8")
56         _, written = write_light_default_config(cfg, overwrite=True)
57         assert written is True
58         assert json.loads(cfg.read_text(encoding="utf-8")) == LIGHT_DEFAULT_CONFIG
59
60     def test_seeded_config_loads_to_truly_local_motion(tmp_path):
61         """The minimal seeded file, run through the loader, expands via the truly-local
62 PRESET to a
63 coherent torch-free/offline config (motion + no Gemini handoff + local storage + no
64 GPU)."""
65         cfg = tmp_path / "config.json"
66         write_light_default_config(cfg)
67         loaded = load_config(cfg)
68         assert loaded.indexing.default_segmenter == "motion"
69         assert loaded.indexing.use_gpu is False
70         assert loaded.deployment.profile == "truly-local"
71         assert loaded.indexing.skip_ai_handoff is True # supplied by the truly-local
72 preset
73         assert loaded.storage.backend == "local" # supplied by the truly-local
74 preset
75
76     # --- scripts/install.py pure logic
77     -----
78
79     @pytest.fixture(scope="module")
80     def install_mod():
81         return _load_script("install.py")
82
83
84     @pytest.fixture(scope="module")
85     def uninstall_mod():
86         return _load_script("uninstall.py")
87
88
89     @pytest.mark.parametrize(

```

```

86     "system, fname, executable",
87     [("Windows", "KhelSutra.bat", False), ("Darwin", "KhelSutra.command", True),
88      ("Linux", "khelsutra.sh", True)],
89 )
90 def test_shortcut_spec_per_os(install_mod, system, fname, executable):
91     home = Path("/home/u/AppHome")
92     name, content, ex = install_mod.shortcut_spec(system, home)
93     assert name == fname and ex is executable
94     # str(home) is OS-native (backslashes on Windows) ? assert against it, not a
hardcoded literal.
95     assert "RALLY_APP_HOME" in content and str(home) in content
96     assert "indexer --app" in content
97
98
99     def test_resolve_method(install_mod):
100         assert install_mod.resolve_method("auto", uv_present=True) == "uv"
101         assert install_mod.resolve_method("auto", uv_present=False) == "pip"
102         assert install_mod.resolve_method("pip", uv_present=True) == "pip" # explicit wins
103         assert install_mod.resolve_method("uv", uv_present=False) == "uv"
104
105
106     def test_install_command_argv(install_mod):
107         assert install_mod.install_command("uv", ".") == ["uv", "tool", "install", "."]
108         pip = install_mod.install_command("pip", "badminton-highlight-indexer")
109         assert pip[1:] == ["-m", "pip", "install", "badminton-highlight-indexer"]
110
111
112     def test_default_os_app_home_per_os(install_mod):
113         win = install_mod.default_os_app_home(system="Windows", env={"APPDATA":
r"C:\Users\u\AppData\Roaming"})
114         assert win.name == "Khelsutra" and "Roaming" in str(win)
115         lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME":
"/home/u/.local/share"})
116         assert str(lin) == str(Path("/home/u/.local/share") / "Khelsutra")
117
118
119     def test_build_manifest_shape(install_mod):
120         home = Path("/home/u/AppHome")
121         m = install_mod.build_manifest(
122             method="uv", app_home=home, shortcut=home / "khelsutra.sh",
123             created=[home / "khelsutra.sh", home / "uninstall_manifest.json"],
timestamp="2026-06-22T00:00:00+00:00",
124         )
125         assert m["package"] == "badminton-highlight-indexer"
126         assert m["install_method"] == "uv"
127         assert str(home / "config.json") in m["user_data"] # user data is tracked
separately (kept on uninstall)
128         assert m["shortcut"] == str(home / "khelsutra.sh")
129         assert m["schema"] == 1
130
131
132     # --- scripts/uninstall.py pure logic
-----
133
134     def test_uninstall_command_argv(uninstall_mod):
135         assert uninstall_mod.uninstall_command("uv", "badminton-highlight-indexer") == \
136             ["uv", "tool", "uninstall", "badminton-highlight-indexer"]
137         pip = uninstall_mod.uninstall_command("pip", "badminton-highlight-indexer")
138         assert pip[1:] == ["-m", "pip", "uninstall", "-y", "badminton-highlight-indexer"]
139
140
141     def test_find_manifest_explicit(uninstall_mod, tmp_path):
142         m = tmp_path / "uninstall_manifest.json"
143         m.write_text("{} ", encoding="utf-8")
144         assert uninstall_mod.find_manifest(str(m)) == m

```

```

145         assert uninstall_mod.find_manifest(str(tmp_path / "nope.json")) is None
146
147
148     def test_install_uninstall_roundtrip_manifest(install_mod, uninstall_mod, tmp_path):
149         """The manifest install.py would write is consumable by uninstall.py's
expectations."""
150         home = tmp_path / "AppHome"
151         m = install_mod.build_manifest(
152             method="pip", app_home=home, shortcut=home / "KhelSutra.bat",
153             created=[home / "KhelSutra.bat", home / "uninstall_manifest.json"],
154             timestamp="2026-06-22T00:00:00+00:00",
155         )
156         mp = home / "uninstall_manifest.json"
157         home.mkdir(parents=True)
158         mp.write_text(json.dumps(m), encoding="utf-8")
159         assert uninstall_mod.find_manifest(str(mp)) == mp
160         loaded = json.loads(mp.read_text(encoding="utf-8"))
161         assert uninstall_mod.uninstall_command(loaded["install_method"],
loaded["package"])[0:3] in (
162             ["uv", "tool", "uninstall"], [install_mod.sys.executable, "-m", "pip"],
163         )
164

```

16. assistant (2026-06-22T01:26:44.060Z)

The DB is inside `output/` (via `resolve\_output\_dir`), so `user\_data: [config.json, .env, output/]` correctly covers the DB. Now let me focus on the safety analysis of `uninstall.py`. The key concern is the app-home removal logic at lines 135-139, and the `\_rm` symlink guard.

Let me look closely at the critical path: when `--purge` is passed, after deleting user_data, the code checks if `app\_home` has any remaining children (excluding `self\_path`) and removes the whole app-home if "empty". Let me trace what `created` contains and whether `user\_data` paths could overlap or whether `app\_home` itself could be removed unsafely.

Let me also check git history / the manifest's `created` for any way a user-data path leaks in, and verify the `self\_path` exclusion logic on the `iterdir()`.

17. user (2026-06-22T01:26:45.725Z)

78e423d docs(pkg): link P2b-install PR #310 in the S7 tracker
a5c1493 feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run default (gate 08)

18. assistant (2026-06-22T01:26:54.512Z)

Let me trace the critical safety scenarios carefully.

****Scenario analysis for the app-home removal (lines 135-141):****

```

```python
if args.purge and app_home.exists():
 try:
 remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
 if not remaining:
 _rm(app_home, dry)
...

```

This only fires with `--purge`. It lists children, excludes `self\_path` (the running uninstaller copy). If the only remaining thing is the running script itself, it removes the whole app-home via `\_rm` ? `rmtree`. This is guarded by `not is\_symlink()` in `\_rm`, and only fires when app-home is effectively empty. Let me think about whether app-home could be a symlink ? `\_rm` handles that (it would `unlink` rather than `rmtree`). That seems OK.

Now the **\*\*most important concern\*\***: could a `user\_data` path NOT be deleted on purge while

`created` deletes data? `created` is `[shortcut, dst\_uninstall, manifest\_path]` ? all installer-made files in app-home. None are user data. Good.

But let me check a subtle issue: **the `--dry-run` + app-home removal interaction**, and whether `\_rm` on app-home during dry-run still lists correctly. More importantly, let me verify the **output/` could contain the source video or be set to an absolute path outside app-home**. Let me check the README diff and how output\_dir flows.

## 19. user (2026-06-22T01:26:55.241Z)

```
diff --git a/README.md b/README.md
index 674e304..0bd46dd 100644
--- a/README.md
+++ b/README.md
@@ -32,6 +32,37 @@ The pipeline is built around three pluggable registries ? validators,
se

?? Setup & Diagnostics

0. Batteries-included install (LIGHT tier ? recommended for end users)
+
+For a download-and-run experience (no `git clone`, no PyTorch, no GPU), use the cross-platform
+installer. It installs the LIGHT tier (torch-free: Gemini / `motion` / cpu-mock), wires a
+desktop launcher, and seeds a sensible default config.
+
+```bash
+# Preferred ? uv puts `indexer` on your PATH in an isolated tool env:
+uv tool install badminton-highlight-indexer # (once published) or: uv tool install .
+
+# Or the cross-platform installer script (uv-if-present, else pip) ? also creates a launcher
+# shortcut + an uninstall manifest:
+python scripts/install.py # add --method pip to force pip, --dry-run
to preview
+```
+
+Then launch the app (the installer's shortcut, or directly):
+
+```bash
+indexer --app # boots a local server (127.0.0.1) and opens the bundled KhelSutra UI
+```
+
+* First run seeds a truly-local default ? detector `motion` (offline, no API key, no GPU, no cloud bills); the in-app setup later switches you to Gemini. State (config, DB, reels) lives
+
+in your OS app-data dir (`%APPDATA%\Khelsutra` / `~/Library/Application Support/Khelsutra` / `~/.local/share/Khelsutra`) when launched via the installer's shortcut.
+
+* FFmpeg is still required (see step 1) ? it is intentionally not redistributed (GPL `libx264`).
+
+* Uninstall cleanly: `python uninstall.py` (dropped next to your data) ? keeps your config/reels
+
+by default; add `--purge` to delete them too.
+
+The native-GPU (WASB) and cloud tiers are separate, opt-in provisioning ? see `docs/PACKAGING_PLAN.md`.
+
1. External System Requirements
The indexer depends on FFmpeg and FFprobe to parse media containers, extract keyframes, and stitch clips.
```

## 20. assistant (2026-06-22T01:27:04.079Z)

Now I have a complete picture. Let me do a hard, focused analysis on each of the 5 questions, and check one subtle but real bug.

**The critical subtle bug ? `created` removal does NOT verify paths are inside app-home.**

The `created` list is consumed at lines 119-122:

```
```python
for p in created:
    if p.resolve() == self_path or p.resolve() == manifest_path.resolve():
        continue
    _rm(p, dry)
```
```

This loop runs on **every** uninstall (not just `--purge`). It deletes whatever paths are in the manifest's `created` list. The manifest is JSON read from disk (line 94) ? it is **fully** attacker/corruption/edit controllable. If `created` contains, say, `"/` or `"C:\Users\me"` or `~/Documents`, `\_rm` will `rmtree` it (it's a dir, not a symlink). There is **no** containment check that `created` entries live under `app\_home`. This is the classic "trust the manifest blindly" data-loss vector.

How realistic is this? The manifest is normally written by `install.py` with safe values. But: (a) a user could hand-edit it, (b) a future installer bug could record a wrong path, (c) the manifest from a **different** install (different app-home) could be passed via `--manifest`. The guard against catastrophic deletion should not depend on the manifest being pristine. Let me verify there is genuinely no containment check anywhere.

## 21. user (2026-06-22T01:27:05.638Z)

```
35: def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
55: default_os_app_home() / MANIFEST_NAME):
99: app_home = Path(manifest.get("app_home", str(manifest_path.parent)))
103: print(f" app-home : {app_home}")
135: if args.purge and app_home.exists():
137: remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
139: _rm(app_home, dry)
144: if self_path.parent == app_home:
145: print(f" (you can delete this script + the app-home folder manually: {app_home})")
```

## 22. assistant (2026-06-22T01:27:20.221Z)

Confirmed: there is **no** containment check anywhere. `created` and `user\_data` paths are deleted as-is from the manifest, with no verification they live under `app\_home`. Let me now verify the remaining concerns and check the `\_rm` symlink behavior on a symlinked dir more carefully, plus whether `output/` being an absolute path outside app-home is a real possibility.

Let me confirm: with the packaged launcher, `RALLY\_APP\_HOME` is set, and `resolve\_output\_dir("output")` ? `/output`. The manifest hardcodes `app\_home / "output"`. So under normal install, `output` is inside app-home. But the user can configure `indexing.output\_dir` to an absolute path (e.g. `D:\MyVideos`), and the engine writes reels there ? yet the manifest still lists only `/output`. That's a **non-issue** for safety\* (purge would just miss it, not over-delete).

Now let me double-check the symlink guard in `\_rm` for the dir case. Line 70: `if path.is\_dir() and not path.is\_symlink()`. A symlink to a directory: `is\_dir()` returns True (follows the link) AND `is\_symlink()` returns True, so the `and not path.is\_symlink()` makes it fall to the `elif` ? `unlink()`, removing only the link. Good ? that guard is correct.

But there's a subtle issue: **shutil.rmtree** with `ignore\_errors=True` (line 73). If app-home contains a symlink to a directory **inside** it, `rmtree` by default does NOT follow symlinks (it removes the link). That's fine. But `ignore\_errors=True` means any failure is silently swallowed ? combined with the `--purge` app-home removal, a partial failure leaves no error. Minor.

Let me check one more real concern: the `self\_path` exclusion when `created` paths are passed through `.resolve()` but `self\_path` exclusion in the `iterdir` uses `.resolve()` too ? consistent. Let me verify the dry-run path for app-home: in dry-run, `\_rm(manifest\_path, dry)` won't actually delete, so `app\_home.exists()` is still true, and `iterdir` lists real children including manifest ? so `remaining` is non-empty and app-home isn't "removed" in the plan. That means **dry-run under-reports**: it won't show the app-home removal that a real run would do (because real run deletes manifest first, emptying the dir). Minor cosmetic but worth noting.

Now let me write up findings. Let me verify the tests don't already cover the containment scenario (they don't ? no test feeds a malicious `created`).

### 23. user (2026-06-22T01:27:21.067Z)

```
112: def test_default_os_app_home_per_os(install_mod):
113: win = install_mod.default_os_app_home(system="Windows", env={"APPDATA":
r"C:\Users\u\AppData\Roaming"})
115: lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME":
"/home/u/.local/share"})
122: method="uv", app_home=home, shortcut=home / "khelsutra.sh",
123: created=[home / "khelsutra.sh", home / "uninstall_manifest.json"],
timestamp="2026-06-22T00:00:00+00:00",
127: assert str(home / "config.json") in m["user_data"] # user data is tracked separately
(kept on uninstall)
152: method="pip", app_home=home, shortcut=home / "KhelSutra.bat",
153: created=[home / "KhelSutra.bat", home / "uninstall_manifest.json"],
```

### 24. assistant (2026-06-22T01:27:31.104Z)

Confirmed ? no test exercises the actual deletion paths (`main`, `\_rm`) with adversarial input; only pure helpers are tested. The deletion orchestration is entirely untested.

I have enough to report. Let me summarize my findings on the 5 questions:

1. **\*\*Default keeps user data?\*\*** YES ? verified. Default uninstall only removes `created` (shortcut/uninstaller-copy/manifest) and prints `user\_data` as kept. Correct.
2. **\*\*Could `--purge`/`created` removal over-delete?\*\*** YES ? the real bug. `created` and `user\_data` are deleted verbatim from a disk-resident, hand-editable JSON manifest with **\*\*no containment check\*\*** that paths live under `app\_home`. A corrupted/edited/foreign manifest can make even a **\*default\*** (non-purge) uninstall `rmtree` an arbitrary directory.
3. **\*\*`\_rm` symlink guard?\*\*** Correct for the symlinked-dir case (falls to `unlink`). But `ignore\_errors=True` silently swallows failures, and there's no too-broad-path guard.
4. **\*\*Could `created` contain a user-data path?\*\*** Not under the current installer (it appends only shortcut/uninstaller/manifest). Low risk in practice, but nothing **\*enforces\*** the `created`/`user\_data` disjointness, so a future installer change could regress silently.
5. **\*\*No-overwrite config seeding?\*\*** Correct ? `write\_light\_default\_config` returns early if the file exists (verified by tests). Protects user/wizard config.

Let me also note the app-home auto-removal and dry-run mismatch.

### 25. user (2026-06-22T01:27:55.691Z)

Structured output provided successfully

### 26. assistant (2026-06-22T01:28:07.906Z)

I reviewed PR #310 (P2b-install) focusing on data-loss/safety, reading `scripts/uninstall.py`, `scripts/install.py` (build\_manifest/user\_data/created), `backend/config/loader.py` (write\_light\_default\_config), `backend/main.py`, `backend/utils/app\_home.py`, and the tests.

Answers to the five questions:

1. **\*\*Default keeps user data ?** YES, correct. **\*\*** A plain `python uninstall.py` removes only `created` (shortcut, copied uninstaller, manifest) and prints `config.json`/`.env`/`output/` as kept.
2. **\*\*Could `--purge`/`created` over-delete ?** YES, the real bug (major). **\*\*** Both `created` (deleted on **\*every\*** run) and `user\_data` (deleted on `--purge`) come from a disk-resident, hand-editable JSON manifest and are passed straight to `rmtree` with **\*\*no check that they live under `app\_home`\*\***. A corrupted, edited, stale, or foreign (`--manifest`) manifest can make even a non-purge uninstall `rmtree` an arbitrary directory.
3. **\*\*`\_rm` symlink guard ?** correct for the symlinked-dir case **\*\*** (`is\_dir()` and not `is\_symlink()`)

falls through to ``unlink``), but there is **\*\*no too-broad/outside-app-home guard\*\***, and ``rmtree(ignore_errors=True)`` silently swallows failures.

4. **\*\*Could ``created`` contain a user-data path ? not today, but unenforced (nit).\*\*** The disjointness of ``created`` vs ``user_data`` is by convention only; no assertion/test prevents a future installer change from regressing a default uninstall into a data-deleting one.

5. **\*\*No-overwrite config seeding ? correct.\*\*** ``write_light_default_config`` returns early when the file exists (covered by tests), preserving a user's config and the future P2c wizard's choices.

I also flagged the ``--purge`` whole-app-home auto-removal sharing the missing containment guard (minor) and a dry-run/real-run divergence for the most destructive step (nit). Note: the deletion orchestration (``main`/`_rm``) is entirely untested ? ``tests/test_p2b_install.py`` only covers pure helpers.

Findings returned via `StructuredOutput` (1 major, 1 minor, 2 nits).